

Monitoramento e Controle de Tráfego Urbano com Semáforos Inteligentes em um Sistema Distribuído

Bruno Braga, Caio Gomes, Kaio Henrique Lucio e Santos,
Lucas Araujo, Nalberth Vieira, Pedro Henrique Bellone, Vitor Alexandre

¹Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

Resumo. *Este trabalho apresenta o desenvolvimento completo de um sistema distribuído autônomo para monitoramento e controle coordenado de semáforos inteligentes em cruzamentos urbanos de quatro vias. A arquitetura evoluiu de um protótipo inicial baseado em gRPC para uma infraestrutura em Malha HTTP (HTTP Mesh) acionada por Express, garantindo o isolamento de processos e a comunicação por passagem de mensagens. O sistema integra Relógios Lógicos de Lamport para ordenação de eventos concorrentes, o Algoritmo Bully para eleição de líder, réplicas com consistência eventual e mecanismos ativos de tratamento de falhas por heartbeat e recuperação automática de estado. Os resultados experimentais e benchmarks obtidos comprovam que o custo de latência da abordagem distribuída é mínimo em comparação com o ganho crítico em tolerância a falhas, resiliência e alta disponibilidade do tráfego urbano.*

Abstract. *This paper presents the development of an autonomous distributed system for coordinated traffic light control in four-way urban intersections. The architecture evolved from an initial gRPC prototype to an HTTP Mesh infrastructure powered by Express, ensuring process isolation and message-passing communication. The system integrates Lamport Logical Clocks for concurrent event ordering, the Bully Algorithm for leader election, eventually consistent replicas, and active fault tolerance through heartbeats and automatic state recovery. Experimental and benchmark results demonstrate that the distributed approach's latency overhead is negligible compared to the critical gains in fault tolerance, resilience, and high availability for urban traffic control.*

1. Introdução e Definição do Problema

Sistemas de trânsito urbano dependem de decisões rápidas e coordenadas para mitigar congestionamentos. Em arquiteturas puramente centralizadas, a queda do servidor principal interrompe a sinalização de múltiplos cruzamentos, gerando gargalos críticos na malha viária. A aplicação de sistemas distribuídos neste cenário surge como uma alternativa para eliminar pontos únicos de falha, descentralizando a inteligência de atuação em múltiplos nós independentes e cooperantes.

O problema abordado consiste no monitoramento distribuído de um cruzamento de quatro vias (*Via A, B, C e D*). Cada via possui contagem flutuante de veículos por minuto (v/m). O desafio reside em coordenar a alternância dos semáforos de forma justa (através de um ciclo padrão *Round-Robin*) ao mesmo tempo em que o sistema deve responder dinamicamente a alertas de tráfego intenso (limiar crítico de 70 veículos). O

cenário exige que múltiplos nós gerenciem suas próprias réplicas do cruzamento, ordenem eventos concorrentes vindos de múltiplos sensores e garantam a continuidade do fluxo viário mesmo se o nó coordenador do cluster sofrer uma pane física repentina.

2. Arquitetura do Sistema e Evolução Tecnológica

A arquitetura do sistema é dividida em três camadas complementares que interagem por meio de comunicação desacoplada e assíncrona.

2.1. Camadas de Borda e Integração

A camada de borda é composta por uma interface Web responsiva em React e Vite, publicada via GitHub Pages. Ela atua como um Atuador de Borda, simulando os sensores físicos instalados nas vias e permitindo a injeção manual de dados de tráfego a partir de computadores ou dispositivos móveis na mesma rede local.

A integração ocorre de forma embutida em cada nó por meio de uma API REST Express. O painel web se comunica com o cluster submetendo métricas de sensores via endpoint público `POST /report`, recebendo de volta o estado sincronizado das vias por meio do método `GET /intersection-status`. O mecanismo de CORS é habilitado de forma aberta (`app.use(cors())`) para viabilizar conexões externas de múltiplos dispositivos de borda simultaneamente.

2.2. Camada de Processamento Distribuído e Malha HTTP

O núcleo de processamento é formado por três nós independentes rodando em processos isolados do Node.js, escutando nas portas lógicas 50051, 50052 e 50053.

Na primeira fase do projeto, a camada de transporte utilizava gRPC e buffers de protocolo (`.proto`). Contudo, testes em ambiente de rede local revelaram restrições severas do subsistema de DNS interno do gRPC para resolução de nomes locais. Para mitigar o problema e garantir a portabilidade, a camada foi migrada para uma **Malha HTTP (*HTTP Mesh*)** baseada em Express. Os nós passaram a se comunicar via requisições HTTP internas trocando payloads formatados em JSON. O isolamento estrito de processos e a natureza de passagem de mensagens foram integralmente preservados, mantendo intactas as propriedades exigidas pelo modelo distribuído.

2.3. Fluxo de Sincronização de Dados

Quando um sensor injeta tráfego em qualquer nó do cluster via `POST /report`, o receptor atua como coordenador momentâneo daquela escrita. Ele incrementa seu relógio lógico de Lamport, atualiza seu dicionário local de memória (`intersectionState`) e propaga os dados em *multicast* para os demais parceiros através da rota interna `POST /internal/report-traffic`. Ao receberem o payload, os nós adjacentes ajustam seus relógios lógicos e atualizam suas respectivas tabelas. Esse processo garante propriedades de **Consistência Eventual**: todas as réplicas convergem de forma transparente para o mesmo estado viário.

3. Algoritmos Distribuídos e Sincronização

3.1. Relógios Lógicos de Lamport

Para resolver a ausência de um relógio físico perfeitamente sincronizado entre os nós, implementou-se o algoritmo de Relógio Lógico de Lamport [1]. Cada nó gerencia uma

variável inteira `lamportClock`. O contador é incrementado localmente a cada novo evento gerado. Nas transmissões de rede, o valor atual do relógio é anexado ao JSON. Ao receber a mensagem, o nó destinatário executa a função de atualização:

$$L_{local} = \max(L_{local}, L_{recebido}) + 1$$

Esse mecanismo impede o entrelaçamento de mensagens e garante que os eventos enviados de diferentes dispositivos concorrentes (como um PC e um celular controlando o tráfego ao mesmo tempo) sejam processados na ordem cronológica causal correta.

3.2. Eleição de Líder via Algoritmo Bully

O cluster necessita de um líder centralizado para ditar a temporização dos semáforos e evitar colisões lógicas. A coordenação é gerida através do Algoritmo Bully [2]. Cada processo possui um identificador fixo (`myId`, de 1 a 3). Caso o líder atual fique indisponível, qualquer nó ativo pode disparar o método `startElection()`, enviando uma requisição de eleição (`POST /internal/election`) para os nós com IDs superiores ao seu. Se nenhum nó de maior prioridade responder dentro do tempo limite (2,5 segundos), o nó proponente assume o controle e executa o método `announceVictory()`, notificando as demais réplicas operantes através da rota `POST /internal/set-coordinator`.

4. Tratamento de Falhas e Resiliência

O tratamento de falhas atua de forma automatizada em três frentes: detecção ativa, reeleição e auto-recuperação pós-reconexão.

4.1. Mecanismo de Heartbeat e Failover

Os nós secundários monitoram a integridade do coordenador por meio de pings periódicos a cada 2 segundos (`HEARTBEAT_INTERVAL`) disparados contra o endpoint `GET /internal/ping`. Se o coordenador sofrer uma queda abrupta, a ausência de resposta gera uma exceção tratada. O nó secundário limpa a referência do líder e invoca imediatamente a rotina de eleição Bully. Chamadas para nós comuns caídos são encapsuladas em blocos `try/catch`, fazendo com que falhas parciais de hardware degradem o sistema de forma graciosa sem interromper os nós remanescentes.

4.2. Sincronização e Recuperação de Estado

Para evitar que um nó retorne ao cluster com dados defasados após uma reinicialização, implementou-se a rotina `syncStateFromPeers()`. Durante o *bootstrap* do servidor, o nó recém-conectado efetua uma requisição de leitura no endpoint interno `GET /internal/state` do primeiro peer disponível. O nó copia integralmente o vetor de tráfego atual das quatro vias e o valor do relógio lógico de Lamport, restabelecendo sua conformidade com o ecossistema distribuído em tempo real.

5. Otimizações de Tráfego e Telemetria

A lógica de controle do cruzamento recebeu melhorias significativas para reproduzir o comportamento de engenharia de tráfego real:

- **Escalonamento por Carga Máxima Global:** O coordenador avalia constantemente o volume total de carros nas quatro vias. Se múltiplas vias ultrapassarem o limiar de 70 veículos, o algoritmo de alocação dinâmica prioriza estritamente aquela com maior saturação real, impedindo o bloqueio injusto de vias críticas.
- **Preempção Suave de Sinais:** Substituiu-se a transição abrupta de sinais (Verde direto para o Vermelho) por um estado de transição segura (`isTransitioning`). Ao ceder prioridade para uma via congestionada, a via aberta atual assume o estado Amarelo obrigatório por 3 segundos, permitindo a vazão segura dos veículos antes do fechamento.
- **Simulação de Vazão Contínua:** Enquanto uma via permanece com o sinal Verde ativo, o servidor executa uma redução lógica de 2 carros por segundo em seu contador, simulando o escoamento contínuo do fluxo rodoviário.
- **Painel de Telemetria Integrado:** Desenvolveu-se um console visual estruturado em formato de tabela (`console.table`). A cada segundo ou alteração de estado, os nós imprimem de forma síncrona o panorama completo do cruzamento (Status, Tempo Restante e Veículos por Via), facilitando processos de auditoria.

6. Resultados Obtidos e Análise de Desempenho

Para mensurar o impacto gerado pela camada de rede distribuída, utilizou-se o script utilitário `benchmark.js` para submeter uma bateria de 50 escritas concorrentes no endpoint `POST /report`. O teste comparou o modelo distribuído proposto (HTTP Mesh com replicação em 3 nós ativos) contra um modelo de referência centralizado operando puramente em memória física local. Os resultados de latência e os critérios operacionais de tolerância a falhas estão consolidados nas tabelas a seguir.

Tabela 1. Análise comparativa de latência de escrita (50 requisições).

Modo de Operação	Tempo Médio	Tempo Mínimo	Tempo Máximo
Distribuído (HTTP Mesh + Propagação)	6,4 ms	4,7 ms	40,0 ms
Centralizado (Referência em Memória)	< 0,1 ms	~ 0 ms	0,02 ms

Tabela 2. Matriz comparativa de resiliência e disponibilidade.

Critério de Avaliação	Modelo Centralizado	Modelo Distribuído (Mesh)
Ponto Único de Falha (SPOF)	Sim (Processo Único)	Não (Réplicas Independentes)
Impacto em Caso de Pane	Interrupção Total do Sistema	Acionamento Automático de Failover
Tempo Médio de Recuperação	Intervenção Manual Humana	< 5 segundos (Bully Election)
Preservação do Estado Viário	Perda Total dos Dados	Integridade Mantida via <i>Multicast</i>

Os testes de estresse práticos validaram a eficácia do sistema: ao encerrar o processo do Nó 3 (Coordenador ativo), o Nó 1 detectou a ausência de pulso após dois ciclos de heartbeat defeituosos, iniciou o escrutínio e o Nó 2 assumiu o controle em menos de 5 segundos. O cruzamento manteve o ciclo de semáforos funcional durante todo o processo de transição. Conforme evidenciado pela Tabela 1, o *overhead* de poucos milissegundos adicionado pela malha HTTP é irrelevante frente ao ganho em robustez e alta disponibilidade demonstrado na Tabela 2.

7. Conclusão

O sistema desenvolvido cumpre com êxito os requisitos de um ambiente distribuído resiliente aplicado ao controle de tráfego urbano. A substituição do gRPC pelo protocolo HTTP Mesh provou que os pilares fundamentais da disciplina — como exclusão mútua, consenso para eleição de líderes e sincronização temporal por ordenação lógica — operam de forma independente da tecnologia de transporte utilizada. A arquitetura garantiu alto desempenho, estabilidade sob concorrência e capacidade de auto-recuperação automatizada sem intervenções manuais, apresentando viabilidade técnica para cenários reais de infraestruturas de cidades inteligentes.

Referências

- [1] LAMPORT, Leslie. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, v. 21, n. 7, p. 558–565, 1978.
- [2] GARCIA-MOLINA, Hector. “Elections in a Distributed Computing System.” *IEEE Transactions on Computers*, v. C-31, n. 1, p. 48–59, 1982.
- [3] TANENBAUM, Andrew S.; VAN STEEN, Maarten. *Distributed Systems: Principles and Paradigms*. 2. ed. Upper Saddle River: Prentice Hall, 2007.